



Lab Activity



Week 8: List Mutability and Memory - References and Alias



Duration: 55 minutes — Work in pairs!

Name: _____ Partner: _____

Lab Goals

By the end of this lab, you will be able to:

- Understand that lists are mutable (can be changed in place)
- Predict how assignment creates references, not copies
- Identify and fix aliasing bugs
- Create independent copies of lists
- Draw memory diagrams to visualize list behavior

Getting Started

- Extract the given [week8_lab.zip](#) file to your desktop
- **KEY CONCEPT:** `list2 = list1` creates a reference, NOT a copy!
- Have paper ready to draw memory diagrams

1 Exercise 1: Understanding Mutability (8 minutes)

Discover the difference between mutable and immutable types!

Open [week8_ex1.py](#) and complete:

```
1  # Experiment 1: String behavior (immutable)
2  str1 = "Hello"
3  str1_id_before = id(str1)
4
5  str1 = str1 + " World"  # Creates NEW string
6  str1_id_after = id(str1)
7
8  print("=== STRINGS (Immutable) ===")
9  print("Before ID:", str1_id_before)
10 print("After ID:", str1_id_after)
11 print("Same object?", str1_id_before == str1_id_after)
12
13 # Experiment 2: List behavior (mutable)
14 list1 = [1, 2, 3]
15 list1_id_before = id(list1)
16
17 list1.append(4)  # Modifies SAME list
18 list1_id_after = id(list1)
19
20 print("\n=== LISTS (Mutable) ===")
```

```

21 print("Before ID:", list1_id_before)
22 print("After ID:", list1_id_after)
23 print("Same object?", list1_id_before == list1_id_after)
24
25 # Practice: Verify mutability
26 numbers = [10, 20, 30]
27 print("\n== Mutation Test ==")
28 print("Original:", numbers, "ID:", id(numbers))
29
30 numbers[1] = ___ # Change middle element
31 numbers._____(40) # Add element
32
33 print("Modified:", numbers, "ID:", id(numbers))
34 print("Did ID change?", ___) # Fill in: Yes or No

```

 **Memory Diagram:** Lists can be changed without creating a new object - this is mutability!

 **Checkpoint:** Can you explain mutability? Show teacher!

2 Exercise 2: References vs Copies (10 minutes)

Learn the crucial difference between assignment and copying!

Open [week8_ex2.py](#) and complete:

```

1 # Experiment: Assignment creates reference
2 original = [1, 2, 3]
3 reference = original # NOT a copy!
4
5 print("== AFTER ASSIGNMENT ==")
6 print("Original:", original)
7 print("Reference:", reference)
8 print("Same object?", original is reference) # True!
9
10 # Modify through reference
11 reference.append(4)
12
13 print("\n== AFTER MODIFYING REFERENCE ==")
14 print("Original:", original) # Also changed!
15 print("Reference:", reference)
16
17 # Why? Draw memory diagram here:
18 #
19 # original --> [1, 2, 3, 4] <-- reference
20 # (Both point to SAME list!)
21
22 # Practice: Predict the output
23 list_a = [5, 10, 15]
24 list_b = list_a
25
26 list_a[0] = ___ # Change first element of list_a
27 print("\n== Practice ==")
28 print("list_a:", list_a)
29 print("list_b:", list_b) # What will this show?
30
31 # Test: Are they the same object?

```

```
32 print("Same object?", list_a == list_b) # Use 'is' operator
```

💡 **Tip:** Use 'is' to check if two variables point to the SAME object!

👯 **Pair Programming:** Switch who types every 5 minutes!

3 Exercise 3: Aliasing Experiments (10 minutes)

Experience aliasing firsthand and learn to recognize it!

Open [week8_ex3.py](#) and complete:

```
1 # Scenario 1: Shopping list aliasing
2 weekly_items = ["milk", "bread", "eggs"]
3 today_shopping = weekly_items # Aliasing!
4
5 print("=== BEFORE MODIFICATION ===")
6 print("Weekly:", weekly_items)
7 print("Today:", today_shopping)
8
9 # Add item to today's list
10 today_shopping.append("cookies")
11
12 print("\n=== AFTER MODIFICATION ===")
13 print("Weekly:", weekly_items) # Surprise!
14 print("Today:", today_shopping)
15 print("Both changed? Why?")
16
17 # Scenario 2: Function aliasing
18 def add_bonus(scores):
19     """Adds 5 bonus points to all scores"""
20     for i in range(len(scores)):
21         scores[i] += 5
22     return scores
23
24 test_scores = [85, 90, 78]
25 print("\n=== FUNCTION TEST ===")
26 print("Before:", test_scores)
27
28 updated = add_bonus(test_scores)
29
30 print("After:", test_scores) # Original changed!
31 print("Returned:", updated)
32 print("Same object?", test_scores == updated) # Fill in 'is'
33
34 # Practice: Fix the aliasing
35 numbers = [1, 2, 3, 4, 5]
36 backup = numbers # BUG: This creates alias!
37
38 numbers[0] = 999
39
40 print("\n=== Aliasing Bug ===")
41 print("Numbers:", numbers)
42 print("Backup:", backup) # Also changed - not a backup!
43
44 # How to fix? We'll learn in next exercise!
```

 **Memory Diagram:** Passing a list to a function passes a reference, so the function can modify the original!

 **Checkpoint:** Can you identify aliasing bugs? Great!

4 Exercise 4: Creating True Copies (9 minutes)

Master the three ways to copy lists properly!

Open `week8_ex4.py` and complete:

```

1 original = [1, 2, 3, 4]
2
3 # Method 1: Slice notation
4 copy1 = original[:]
5 print("=== Method 1: Slice [:] ===")
6 print("Original:", original)
7 print("Copy1:", copy1)
8 print("Same object?", original == copy1) # False!
9
10 # Test independence
11 copy1[0] = 999
12 print("After changing copy:")
13 print("Original:", original) # Unchanged!
14 print("Copy1:", copy1)
15
16 # Method 2: list() constructor
17 original = [1, 2, 3, 4] # Reset
18 copy2 = list(original)
19 print("\n=== Method 2: list() ===")
20 print("Same object?", original == copy2) # False!
21
22 # Method 3: copy() method
23 copy3 = original.____() # Fill in method name
24 print("\n=== Method 3: .copy() ===")
25 print("Same object?", original == copy3) # False!
26
27 # Practice: Fix the backup problem
28 print("\n=== FIXED BACKUP ===")
29 numbers = [1, 2, 3, 4, 5]
30 backup = numbers.____() # Use proper copy!
31
32 numbers[0] = 999
33 numbers.append(6)
34
35 print("Numbers:", numbers)
36 print("Backup:", backup) # Now it's safe!

```

 **Tip:** All three methods create shallow copies. For nested lists, use `copy.deepcopy()`!

5 Exercise 5: Memory Diagram Practice (10 minutes)

Visualize what's happening in memory!

Open [week8_ex5.py](#) and complete:

```

1  # Scenario: Predict then verify
2
3  # Code:
4  a = [10, 20]
5  b = a
6  c = a.copy()
7  b.append(30)
8
9  # BEFORE RUNNING: Draw memory diagram on paper
10 # Show: Which lists share memory? Which are independent?
11
12 print("=== Results ===")
13 print("a:", a)
14 print("b:", b)
15 print("c:", c)
16
17 print("\n=== Identity Checks ===")
18 print("a is b:", a is b) # True or False?
19 print("a is c:", a is c) # True or False?
20 print("b is c:", b is c) # True or False?
21
22 print("\n=== Equality Checks ===")
23 print("a == b:", a == b) # True or False?
24 print("a == c:", a == c) # True or False?
25
26 # Challenge: Complex scenario
27 print("\n=== Challenge ===")
28 x = [1, 2, 3]
29 y = x
30 x = x + [4] # This creates NEW list!
31
32 print("x:", x)
33 print("y:", y)
34 print("x is y:", x is y) # What happened?
35
36 # Explanation:
37 # x + [4] creates a NEW list
38 # So x now points to new list, y still points to old list

```

 **Memory Diagram:** Use 'is' for identity (same object), use '==' for equality (same values)!

 **Checkpoint:** Can you draw memory diagrams? Excellent!

6 Exercise 6: Debugging Aliasing Bugs (8 minutes)

Find and fix common aliasing mistakes!

Open [week8_ex6.py](#) and complete:

```

1  # Bug 1: Unintended sharing
2  def reset_scores(scores):
3      """Sets all scores to 0"""
4      for i in range(len(scores)):
5          scores[i] = 0

```

```

6     return scores
7
8 student_scores = [85, 90, 78]
9 print("=== Bug 1 ===")
10 print("Before:", student_scores)
11
12 # User wants a reset version but keep original
13 reset_version = reset_scores(student_scores)
14
15 print("After:", student_scores) # BUG: Original changed!
16 print("Reset:", reset_version)
17
18 # FIX: Pass a copy to the function
19 student_scores = [85, 90, 78] # Reset
20 reset_version = reset_scores(student_scores.____()) # Copy!
21 print("\nFixed:")
22 print("Original:", student_scores) # Safe!
23 print("Reset:", reset_version)
24
25 # Bug 2: Default argument aliasing
26 def add_item(item, shopping_list=[]): # BUG: Mutable default!
27     shopping_list.append(item)
28     return shopping_list
29
30 # This behaves strangely!
31 list1 = add_item("milk")
32 list2 = add_item("bread") # Both will share same list!
33
34 print("\n=== Bug 2 ===")
35 print("List1:", list1)
36 print("List2:", list2) # Both have milk AND bread!
37
38 # FIX: Use None as default
39 def add_item_fixed(item, shopping_list=None):
40     if shopping_list is None:
41         shopping_list = []
42     shopping_list.append(item)
43     return shopping_list
44
45 list1 = add_item_fixed("milk")
46 list2 = add_item_fixed("bread")
47 print("\nFixed:")
48 print("List1:", list1) # Only milk
49 print("List2:", list2) # Only bread

```

✔ **Checkpoint:** Aliasing bugs fixed? You're a debugging pro!

7 Lab Summary

✔ **Checkpoint:** Final checkpoint - make sure you have:

- ☐ Understood that lists are mutable
- ☐ Recognized that assignment creates references

- ☐ Identified aliasing scenarios
- ☐ Created independent copies with three methods
- ☐ Drew memory diagrams to visualize behavior
- ☐ Used 'is' vs '==' correctly
- ☐ Fixed aliasing bugs
- ☐ Worked effectively with your partner

Reflection (2 minutes)

Rate your understanding of today's concepts:

Concept	😞 Need Help	😬 Getting It	😊 Got It!
List mutability	☐	☐	☐
References vs copies	☐	☐	☐
Identifying aliasing	☐	☐	☐
Memory diagrams	☐	☐	☐

When would you use `.copy()` in your programs?

What's the difference between 'is' and '=='?

😊 Amazing! You understand how lists work in memory!